

Watchly

Jalon 4 — Conception de l'application & Architecture

Champ	Valeur
Auteur	MARTIN Gabriel
Formation	CDA — Concepteur Développeur d'Applications
École	IPSSI · [PRS & DIS 26.1] BD3 26.2 · Session Novembre 2025
Date de rendu	30 avril 2026
Version	1.0
Jalon précédent	Jalon 3 — Modélisation BDD (31/03/2026)
Jalon suivant	Jalon 5 — Développement, Sécurité & Tests (29/05/2026)

CDA — Concepteur Développeur d'Applications

Table des matières

Table des matières	2
I. Introduction	3
I.1 Rappel du périmètre fonctionnel	3
II. Diagrammes de cas d'utilisation	4
II.1 Acteurs du système	4
II.2 Diagramme de cas d'utilisation	4
II.3 Description des cas d'utilisation principaux	6
III. Diagrammes de séquence	7
III.1 Authentification JWT (connexion)	7
III.2 Ajout d'un film à la collection	8
III.3 Notation d'un film	9
IV. Diagramme de classes	10
IV.1 Vue d'ensemble	10
IV.2 Description des entités	10
IV.3 Description des controllers	11
IV.4 Description des services	11
V. Architecture multi-couches	12
V.1 Schéma d'architecture	12
V.2 Pattern MVC avec Symfony	13
V.3 Architecture n-tiers	13
V.4 Docker Compose – Orchestration	13
V.5 Séparation des responsabilités et bonnes pratiques	14
V.6 Composants externes et bibliothèques	14
VI. Conclusion et perspectives	15

I. Introduction

Ce document constitue le livrable du Jalon 4 du projet Watchly, réalisé dans le cadre de la formation CDA (Concepteur Développeur d'Applications) à l'IPSSI. Il porte sur la conception technique de l'application et la description de son architecture logicielle.

Watchly est une application web de gestion de collections de films, construite selon une architecture découplée : une API RESTful développée avec le framework Symfony (PHP 8+) côté back-end, consommée par une Single Page Application React côté front-end. L'application intègre l'API TMDB (The Movie Database) comme source de données films et utilise le bundle LexikJWT pour l'authentification par tokens JWT.

Ce jalon a pour objectif de formaliser l'ensemble de la conception UML (cas d'utilisation, diagrammes de séquence, diagramme de classes) ainsi que la description de l'architecture multi-couches (pattern MVC, architecture n-tiers, séparation des responsabilités). Ces livrables serviront de fondation pour le développement effectif qui débutera en parallèle et se poursuivra lors des jalons 5 et 6.

I.1 Rappel du périmètre fonctionnel

Les fonctionnalités principales de Watchly, définies dans le CDCF (Jalon 1), sont les suivantes :

Fonctionnalité	Description
Authentification	Inscription, connexion via JWT, gestion du profil utilisateur
Recherche de films	Recherche dans le catalogue TMDB via l'API externe, consultation des fiches films
Collection personnelle	Ajout/suppression de films à sa collection avec trois statuts : Watchlist (à voir), Watched (vu) et Favorite (coup de cœur). Notation personnelle (1 à 5 étoiles) pour les films vus.
Notation et avis	Attribution d'une note (1–5) et rédaction d'un avis textuel optionnel sur les films vus
Back-office	Interface d'administration pour la gestion des utilisateurs et statistiques

II. Diagrammes de cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous offre une vue d'ensemble des interactions entre les acteurs et le système Watchly. Trois types d'acteurs ont été identifiés : le Visiteur (non authentifié), l'Utilisateur (authentifié) et l'Administrateur. L'API TMDB apparaît comme un acteur secondaire externe, nécessaire aux cas d'utilisation liés à la recherche et à l'ajout de films.

II.1 Acteurs du système

Acteur	Type	Description
Visiteur	Primaire	Utilisateur non authentifié, accède aux pages publiques (inscription, connexion)
Utilisateur	Primaire	Utilisateur connecté, peut gérer sa collection (watchlist, films vus, favoris), noter des films et rédiger des avis
Administrateur	Primaire	Rôle privilégié permettant la gestion des comptes et la consultation des statistiques
API TMDB	Secondaire	Service externe fournissant les données cinématographiques (métadonnées, affiches, etc.)

II.2 Diagramme de cas d'utilisation

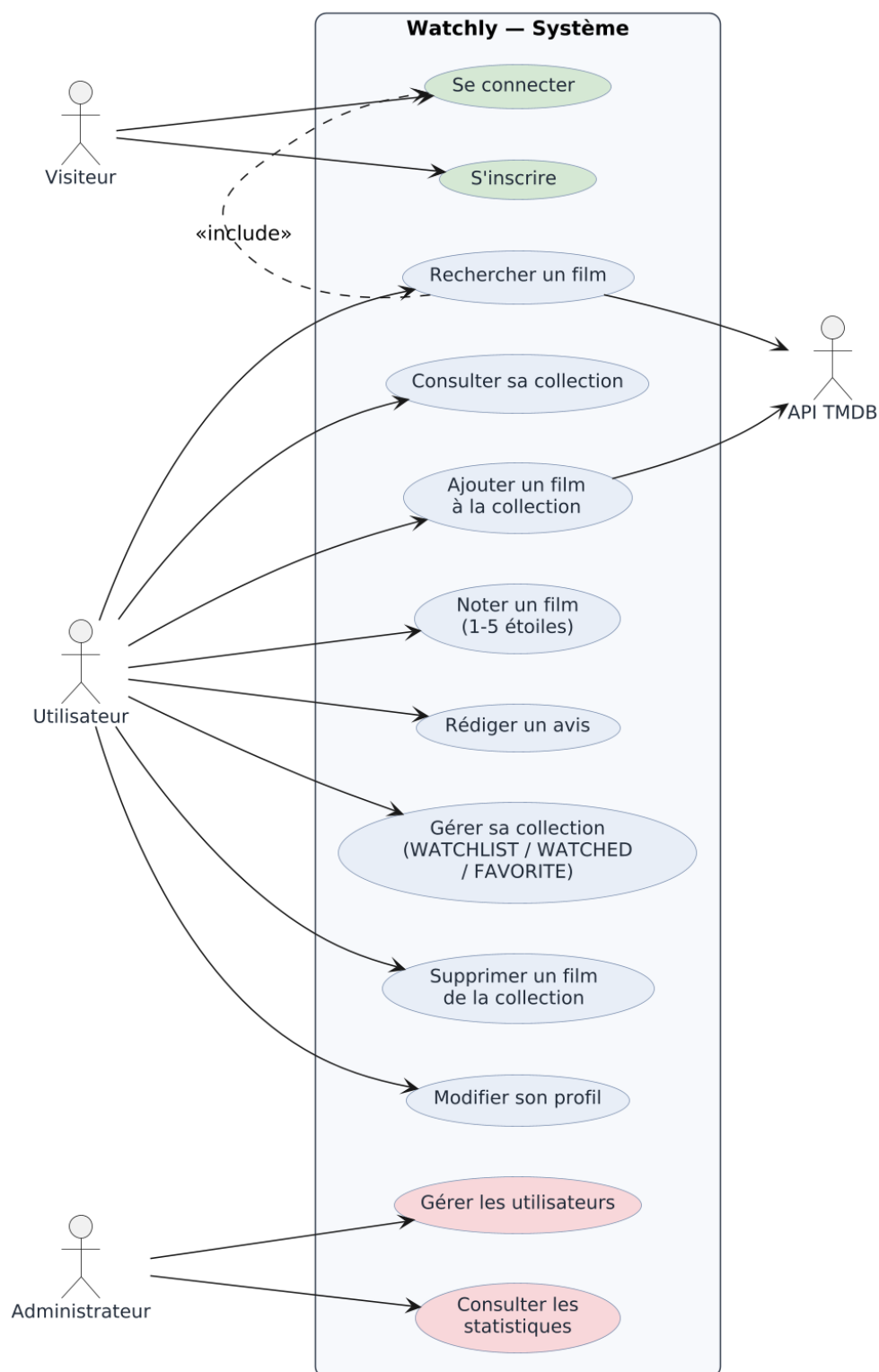


Figure 1 – Diagramme de cas d'utilisation de Watchly

II.3 Description des cas d'utilisation principaux

Cas d'utilisation	Acteur(s)	Précondition	Scénario principal
S'inscrire	Visiteur	Aucune	Le visiteur remplit le formulaire (email, mot de passe, pseudo). Le système valide les données, hache le mot de passe et crée le compte.
Se connecter	Visiteur	Compte existant	Le visiteur saisit ses identifiants. Le système vérifie le mot de passe et retourne un token JWT.
Rechercher un film	Utilisateur, API TMDB	Authentifié	L'utilisateur saisit un terme de recherche. Le système interroge l'API TMDB et affiche les résultats.
Ajouter un film	Utilisateur, API TMDB	Authentifié	L'utilisateur sélectionne un film. Le système récupère les métadonnées TMDB et l'ajoute à la collection.
Noter un film	Utilisateur	Film dans la collection	L'utilisateur attribue une note (1–5) et peut rédiger un avis textuel. La moyenne est recalculée.
Gérer la collection	Utilisateur	Authentifié	L'utilisateur bascule le statut d'un film entre WATCHLIST (à voir), WATCHED (vu) et FAVORITE (coup de cœur). Un favori implique que le film a été vu (RG-04).
Gérer les utilisateurs	Administrateur	Rôle ROLE_ADMIN	L'administrateur peut consulter, désactiver ou supprimer des comptes utilisateurs.

III. Diagrammes de séquence

Les diagrammes de séquence suivants détaillent le flux d'exécution des trois cas d'utilisation les plus représentatifs de l'application. Ils mettent en évidence les échanges entre les composants (front-end React, controllers Symfony, services métier, repositories Doctrine et systèmes externes).

III.1 Authentification JWT (connexion)

Ce diagramme illustre le processus d'authentification d'un utilisateur. La requête POST envoyée depuis le front-end React est traitée par l'AuthController, qui délègue la vérification des identifiants au UserRepository via Doctrine. Si les identifiants sont valides, le bundle LexikJWT génère un token signé retourné au client. Ce token sera ensuite inclus dans le header Authorization de chaque requête subséquente.

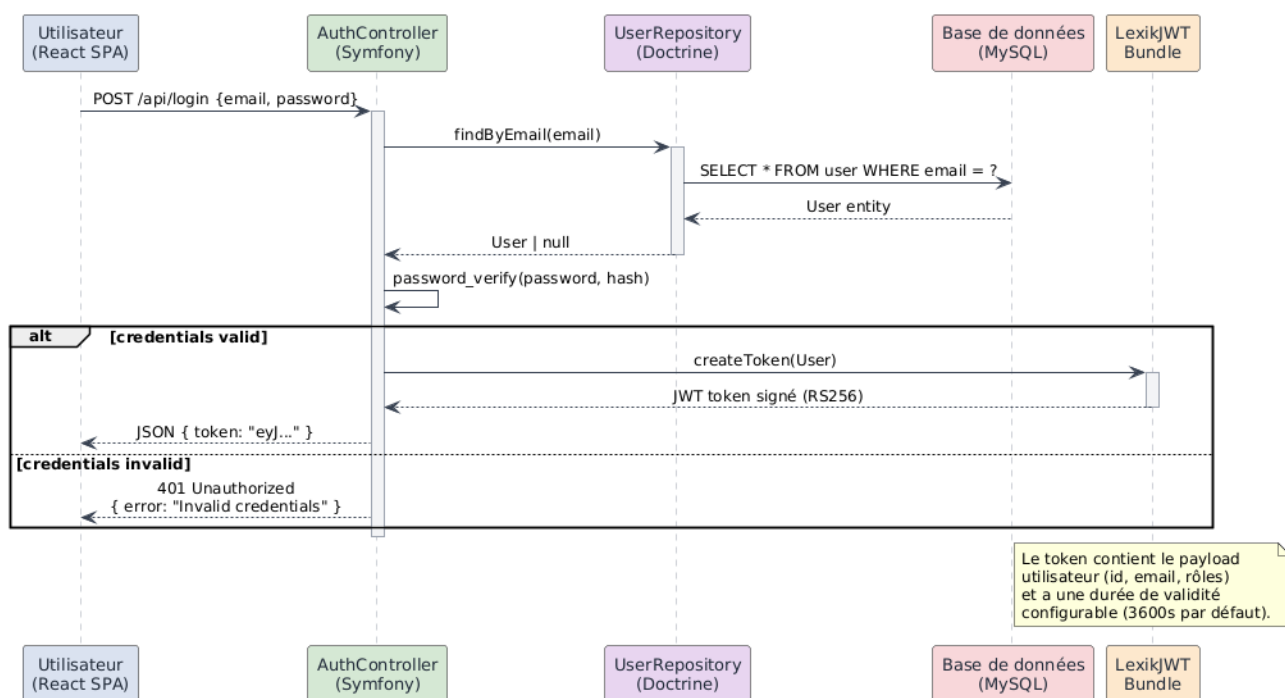


Figure 2 – Diagramme de séquence : Authentification JWT

Le token JWT contient le payload utilisateur (id, email, rôles) et a une durée de validité configurable (par défaut 3600 secondes). Le refresh token peut être implémenté via le bundle `GesdinetJWTRefreshToken` pour une meilleure expérience utilisateur.

III.2 Ajout d'un film à la collection

Ce scénario montre l'interaction entre plusieurs couches du système lors de l'ajout d'un film. Le FilmController vérifie d'abord l'authentification JWT via le middleware de Symfony, puis délègue la récupération des métadonnées au TMDBService. Le film est ensuite créé ou récupéré en base locale (pattern find-or-create) avant d'être associé à la collection de l'utilisateur.

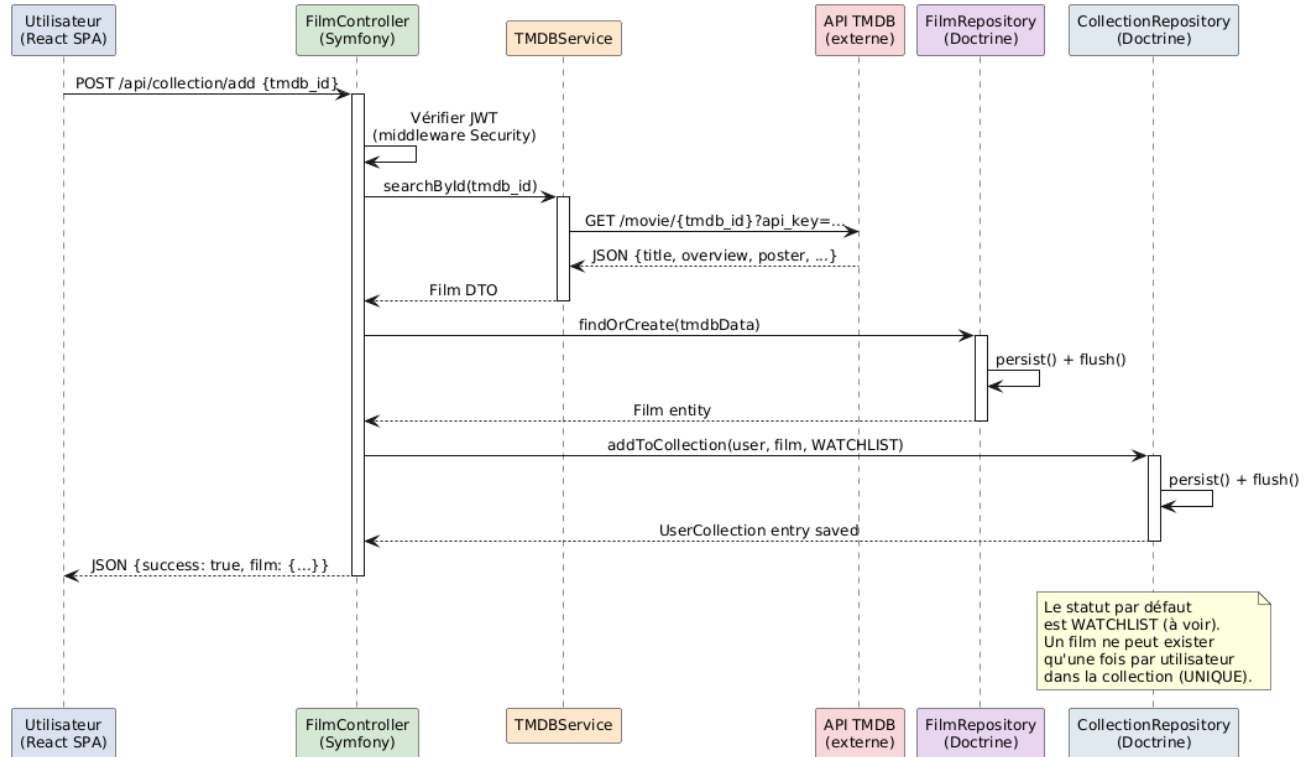


Figure 3 – Diagramme de séquence : Ajout d'un film à la collection

III.3 Notation d'un film

La notation d'un film suit un flux plus simple. Le ReviewController reçoit la requête PUT avec la note et l'avis textuel optionnel, valide les données, puis crée ou met à jour l'entité Review via le repository. La moyenne du film est recalculée après chaque notation et retournée dans la réponse JSON.

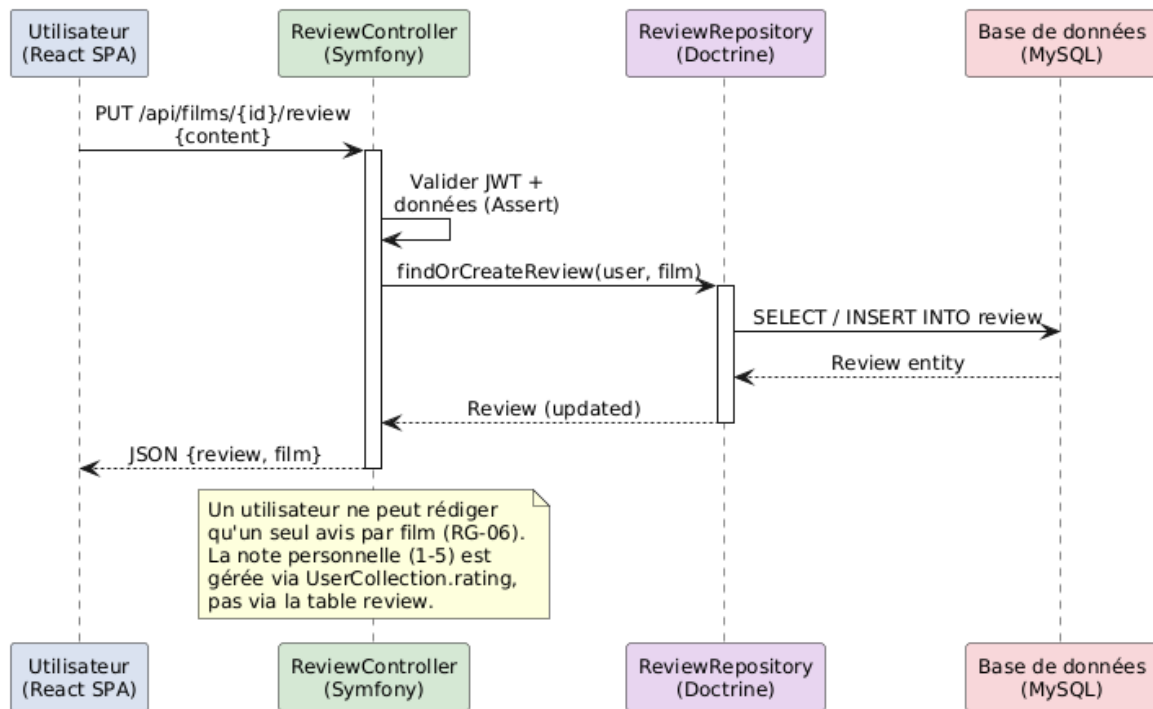


Figure 4 – Diagramme de séquence : Notation d'un film

IV. Diagramme de classes

Le diagramme de classes ci-dessous présente la structure interne de l'application Watchly, organisée en trois couches principales : les entités Doctrine (modèle de données), les controllers (point d'entrée API) et les services (logique métier et intégrations externes).

IV.1 Vue d'ensemble

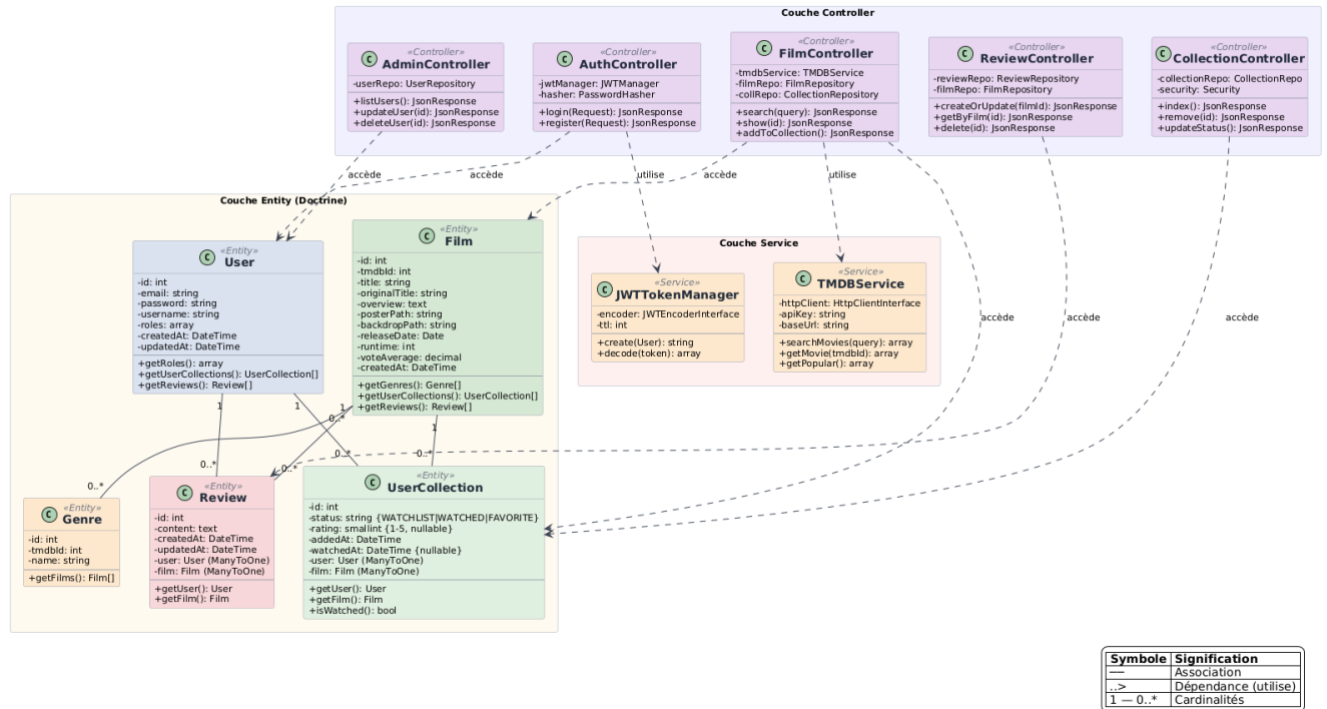


Figure 5 – Diagramme de classes de Watchly

IV.2 Description des entités

Entité	Rôle	Relations principales
User	Représente un utilisateur inscrit (email, mot de passe haché, rôles)	Possède 0..* UserCollection, possède 0..* Review
Film	Film référencé localement, importé depuis TMDB (titre, synopsis, affiche, genres)	Présent dans 0..* UserCollection, possède 0..* Review, associé à 0..* Genre (ManyToMany via film_genre)
UserCollection	Association entre un User et un Film avec statut (WATCHLIST, WATCHED, FAVORITE), note personnelle (1–5) et dates d'ajout/visionnage	Appartient à 1 User, référence 1 Film
Review	Avis textuel rédigé par un User sur un Film (contenu, dates de création/modification)	Appartient à 1 User, référence 1 Film (unique par couple User/Film)
Genre	Catégorie cinématographique importée depuis TMDB (action, drame, comédie, etc.)	Associé à 0..* Film (ManyToMany via table film_genre gérée par Doctrine)

IV.3 Description des controllers

Les controllers Symfony constituent le point d'entrée de l'API REST. Chaque controller est responsable d'un domaine fonctionnel et suit le principe de responsabilité unique (SRP). Les routes sont définies via les attributs PHP 8 (`#[Route]`) et les réponses sont systématiquement au format JSON.

Controller	Routes principales	Dépendances injectées
AuthController	POST /api/login, POST /api/register	JWTTokenManagerInterface, UserPasswordHasherInterface, UserRepository
FilmController	GET /api/films/search, GET /api/films/{id}, POST /api/collection/add	TMDBService, FilmRepository, CollectionRepository
CollectionController	GET /api/collection, DELETE /api/collection/{id}, PATCH /api/collection/{id}	CollectionRepository, Security (current user)
ReviewController	PUT /api/films/{id}/review, GET /api/films/{id}/reviews	ReviewRepository, FilmRepository
AdminController	GET /api/admin/users, PATCH /api/admin/users/{id}, DELETE /api/admin/users/{id}	UserRepository, Security (ROLE_ADMIN)

IV.4 Description des services

Service	Responsabilité	Dépendances
TMDBService	Encapsule les appels HTTP vers l'API TMDB (recherche, détails film, films populaires). Gère la clé API via les variables d'environnement.	HttpClientInterface (Symfony HttpClient)
JWTTokenManager	Génération et validation des tokens JWT via le bundle LexikJWT. Configuré avec une paire de clés RSA (config/jwt/).	JWTEncoderInterface (LexikJWT)

V. Architecture multi-couches

V.1 Schéma d'architecture

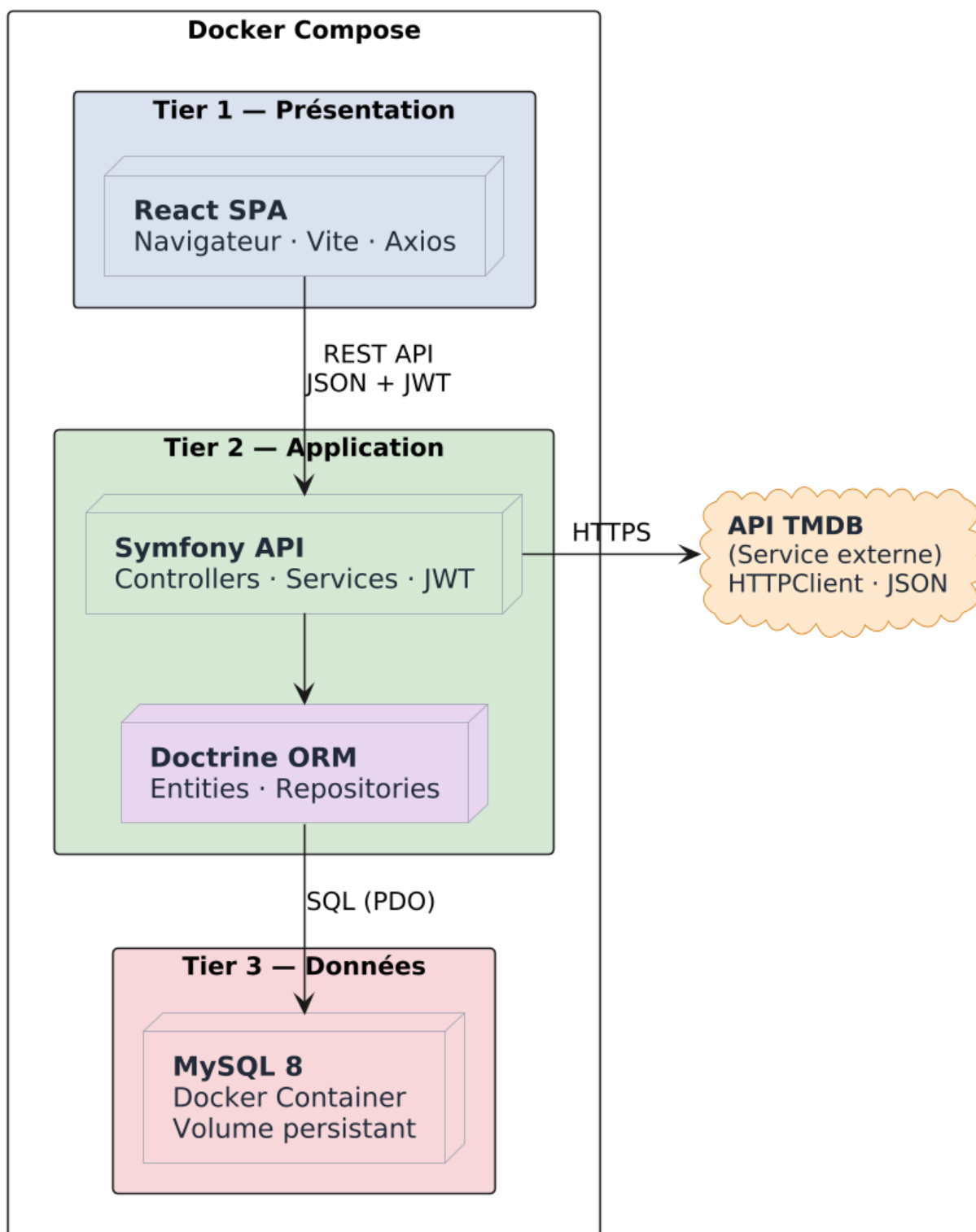


Figure 6 – Architecture n-tiers de Watchly

V.2 Pattern MVC avec Symfony

L'application back-end suit le pattern Model-View-Controller (MVC), adapté au contexte d'une API REST headless où la couche « Vue » est déportée vers le front-end React :

Couche MVC	Implémentation Watchly	Responsabilité
Model	Entités Doctrine (User, Film, Genre, UserCollection, Review) + Repositories	Modélisation du domaine métier et accès aux données. Les repositories encapsulent les requêtes DQL/SQL via l'ORM Doctrine.
Controller	Controllers Symfony (AuthController, FilmController, CollectionController, ReviewController, AdminController)	Réception des requêtes HTTP, validation des entrées, orchestration des appels aux services et repositories, retour des réponses JSON.
View	Application React SPA (composants, pages, hooks)	Rendu de l'interface utilisateur, gestion de l'état local (React state/context), consommation de l'API via Axios.

La couche Vue est entièrement gérée par React, ce qui permet une séparation stricte des responsabilités front-end / back-end et favorise la réutilisabilité de l'API.

V.3 Architecture n-tiers

L'architecture de Watchly se décompose en trois tiers logiques, chacun déployé dans un conteneur Docker séparé :

Tier	Composant	Technologie	Conteneur Docker
Tier 1 – Présentation	Application React SPA	React 18, Vite, Axios, CSS (design system Watchly)	Node/Nginx (build statique servi par Nginx)
Tier 2 – Application	API Symfony + Services métier	Symfony 7, PHP 8.4, LexikJWT, Doctrine ORM	PHP-FPM + Nginx
Tier 3 – Données	Base de données relationnelle	MySQL 8	MySQL officiel

La communication entre les tiers s'effectue via des protocoles standardisés : le front-end communique avec le back-end via des requêtes HTTP REST (JSON) authentifiées par token JWT dans le header Authorization. Le back-end communique avec la base de données via PDO (encapsulé par Doctrine). Le service TMDBService effectue des appels HTTPS sortants vers l'API TMDB.

V.4 Docker Compose – Orchestration

L'ensemble de l'infrastructure est orchestré via Docker Compose. Chaque service est isolé dans son propre conteneur, ce qui garantit la reproductibilité de l'environnement et facilite le déploiement. La configuration prévue est la suivante :

Service	Image	Ports	Volumes / Notes
app (Symfony)	php:8.4-fpm + Nginx	8080:80	Code source monté, .env pour config

db (MySQL)	mysql:8	3306:3306	Volume persistant pour les données
front (React)	node:20 (build) + nginx:alpine	3000:80	Build statique servi par Nginx

V.5 Séparation des responsabilités et bonnes pratiques

La conception de Watchly respecte les principes SOLID et les bonnes pratiques de développement PHP/Symfony :

Principe	Application dans Watchly
Single Responsibility (SRP)	Chaque classe a une responsabilité unique : les controllers orchestrent, les services encapsulent la logique métier, les repositories gèrent l'accès aux données.
Dependency Injection	Symfony injecte automatiquement les dépendances (services, repositories) via l'autowiring. Aucune instantiation manuelle dans les controllers.
Sécurité des secrets	Les clés API (TMDB), les secrets JWT et les identifiants de BDD sont stockés dans le fichier .env (non versionné) et accédés via les paramètres Symfony.
Conventions PSR	Le code PHP suit les standards PSR-4 (autoloading), PSR-12 (coding style) et utilise les attributs PHP 8 pour le routing (<code>#[Route]</code>).
DRY / KISS	Les fonctionnalités répétées (appels TMDB, validation JWT) sont centralisées dans des services réutilisables. Les réponses JSON suivent un format standardisé.

V.6 Composants externes et bibliothèques

Le projet s'appuie sur plusieurs bundles et bibliothèques pour accélérer le développement et garantir la robustesse :

Composant	Rôle	Intégration
LexikJWTAuthenticationBundle	Authentification stateless par tokens JWT (génération, validation, refresh)	Configuré dans security.yaml, firewall « api » avec clés RSA dans config/jwt/
Doctrine ORM	Mapping objet-relationnel, gestion des entités et migrations de base de données	Entités annotées avec attributs PHP 8, migrations gérées via doctrine:migrations
Symfony HttpClient	Client HTTP natif pour les appels à l'API TMDB	Injecté dans TMDBService via HttpClientInterface
Symfony Serializer	Sérialisation/désérialisation des entités en JSON	Utilisé dans les controllers pour formater les réponses API
Symfony Validator	Validation des données entrantes (contraintes sur les entités et les DTOs)	Attributs Assert sur les entités (<code>#[Assert\NotBlank]</code> , <code>#[Assert\Email]</code> , etc.)

NelmioCorsBundle	Configuration des en-têtes CORS pour autoriser les requêtes cross-origin depuis React	Configuré dans nelmio_cors.yaml (origins, headers, methods autorisés)
React 18 + Vite	Framework front-end pour la SPA, bundler rapide en développement	Application autonome communiquant avec l'API via Axios
Axios	Client HTTP côté front-end avec intercepteurs pour le token JWT	Intercepteur global ajoutant le header Authorization: Bearer {token}

VI. Conclusion et perspectives

Ce dossier de conception technique formalise l'ensemble de l'architecture et de la structure interne de Watchly. Les diagrammes UML produits (cas d'utilisation, séquence, classes) offrent une vision claire du fonctionnement de l'application et serviront de référence tout au long du développement.

L'architecture découplée choisie (API Symfony + SPA React) permet une séparation nette des responsabilités et facilite l'évolutivité de chaque couche indépendamment. L'utilisation de Docker garantit la portabilité et la reproductibilité de l'environnement de développement à la production.

Le prochain jalon (Jalon 5 – mai 2026) portera sur le développement effectif de l'application, l'implémentation des mesures de sécurité (OWASP Top 10), la mise en place de la pipeline CI/CD et la rédaction du rapport de tests automatisés. Les diagrammes présentés dans ce document seront mis à jour si l'implémentation révèle des ajustements nécessaires.